Task 1: Guided Activity, Library item model

```swift
enum BookStatus {
    case available
    case borrowed(by: String, dueIns: Int)
    case reserved(for: String)

    var label: String {
        switch self {
        case .available:
            return "Available"
        case .borrowed(let name, let days):
            return "Borrowed by \(name), due in \(days) days"
        case .reserved(let name):
            return "Reserved for \(name)"
        }
    }
}
struct Book {
    var title: String
    var status: BookStatus = .available
}
class Member {
    var name: String
    var borrowedBooks: [String]

    init(name: String, borrowedBooks: [String] = []) {
        self.name = name
        self.borrowedBooks = borrowedBooks
    }
}

var book1 = Book(title: "Swift Programming")
var book2 = book1

book2.status = .borrowed(by: "Sasmi", dueIns: 5)

print("Book 1:", book1.status.label)
print("Book 2:", book2.status.label)

let member1 = Member(name: "Sasmi")
let member2 = member1

member2.name = "Sowmi"

print("Member 1:", member1.name)
print("Member 2:", member2.name)
```

Output:

```
Book 1: Available
Book 2: Borrowed by Sasmi, due in 5 days
Member 1: Sowmi
Member 2: Sowmi
```

Task 2: Practice Task, Shapes and areas

```swift
enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    case triangle(base: Double, height: Double)

    var area: Double {
        switch self {
        case .circle(let radius):
            return Double.pi * radius * radius
        case .rectangle(let width, let height):
            return width * height
        case .triangle(let base, let height):
            return 0.5 * base * height
        }
    }
}

let circle = Shape.circle(radius: 5)
let rectangleShape = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 8, height: 3)

print("Circle area:", circle.area)
print("Rectangle area:", rectangleShape.area)
print("Triangle area:", triangle.area)
struct Rectangle {
    var width: Double
    var height: Double

    var perimeter: Double {
        return 2 * (width + height)
    }

    mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}

var original = Rectangle(width: 4, height: 3)
var copy = original

copy.scale(by: 2)

print("Original rectangle:", original.width, original.height)
print("Original perimeter:", original.perimeter)

print("Scaled rectangle:", copy.width, copy.height)
print("Scaled perimeter:", copy.perimeter)
```

Output:

```
Circle area: 78.53981633974483
Rectangle area: 24.0
Triangle area: 12.0
Original rectangle: 4.0 3.0
Original perimeter: 14.0
Scaled rectangle: 8.0 6.0
Scaled perimeter: 28.0
```

Task 3: Challenge Task

```swift
struct Counter {
    var value = 0

    mutating func increment() {
        value += 1
    }
}
class Box {
    var value = 0

    func increment() {
        value += 1
    }
}
var c = Counter()
c.increment()
print("Counter:", c.value)

let b = Box()
b.increment()
print("Box:", b.value)

let first = Box()
let second = first

second.value = 10

print("First value:", first.value)
print("Second value:", second.value)
print("Same object:", first === second)
```

Output:

```
Counter: 1
Box: 1
First value: 10
Second value: 10
Same object: true
```